rCore-Tutorial 第二章 vs 第三章 代码差异分析

概述

第三章（ch3-coop，协作式调度）在第二章（ch2，批处理系统）的基础上，引入了**任务管理**和**协作式调度**机制，实现了多个应用程序并发执行的能力。

一、文件结构变化

1.1 删除的文件

文件	说明
`os/src/batch.rs`	第二章的批处理子系统，在第三章被完全移除

1.2 新增的文件

文件	说明
`os/src/config.rs`	配置常量（从 batch.rs 中提取）
`os/src/loader.rs`	应用加载器（从 batch.rs 中提取并重构）
`os/src/task/mod.rs`	任务管理器主模块
`os/src/task/task.rs`	任务控制块（TCB）定义
`os/src/task/context.rs`	任务上下文结构
`os/src/task/switch.S`	上下文切换汇编代码

1.3 主要修改的文件

文件	主要变化
`os/src/main.rs`	从 batch 子系统切换到 task 子系统
`os/src/trap/mod.rs`	异常处理从 panic 改为切换到下一个任务
`os/src/syscall/mod.rs`	新增 sys_yield 系统调用
`os/src/syscall/process.rs`	sys_exit 实现改为任务切换
`user/src/syscall.rs`	新增 sys_yield 函数
`user/src/bin/*`	测试程序从单独测试改为协作式并发测试

二、核心架构变化

2.1 应用程序加载方式变化

第二章（batch.rs）：

- 应用程序串行加载，一次只加载一个应用
- 应用加载到固定地址 `0x80400000`
- 只有一组内核栈和用户栈

```
// 第二章：单应用加载
pub fn load_app(&self, app_id: usize) {
    // 加载单个应用到固定地址
    unsafe {
        core::slice::from_raw_parts_mut(APP_BASE_ADDRESS as *mut u8,
APP_SIZE_LIMIT)
            .fill(0);
        // ... 加载应用
    }
}
```

第三章（loader.rs）：

- 应用程序批量预加载，所有应用同时加载到不同内存区域
- 每个应用有独立的内存基地址：`APP_BASE_ADDRESS + app_id * APP_SIZE_LIMIT`
- 每个应用有独立的内核栈和用户栈

```
// 第三章：多应用预加载
pub fn load_apps() {
    for i in 0..num_app {
        let base_i = get_base_i(i);  // 每个应用独立地址
        // ... 加载所有应用
    }
}

fn get_base_i(app_id: usize) -> usize {
    APP_BASE_ADDRESS + app_id * APP_SIZE_LIMIT
}
```

2.2 任务管理机制

第二章：批处理系统

- 应用程序按顺序执行，一次只运行一个
- 应用执行完毕后加载下一个
- 无任务切换概念

```
// 第二章：顺序执行
batch::run_next_app();  // 运行下一个应用
```

第三章：协作式任务调度

- 引入 `TaskControlBlock`（任务控制块）
- 引入 `TaskStatus`（任务状态：UnInit/Ready/Running/Exited）
- 引入 `TaskContext`（任务上下文）
- 引入 `TaskManager`（全局任务管理器）

```
// 第三章：任务控制块
pub struct TaskControlBlock {
    pub task_status: TaskStatus,
    pub task_cx: TaskContext,
}

// 第三章：任务状态
pub enum TaskStatus {
    UnInit,   // 未初始化
    Ready,    // 就绪
    Running,  // 运行中
    Exited,   // 已退出
}
```

2.3 上下文切换机制

第二章：无上下文切换

- 直接通过 `__restore` 跳转到用户态
- 不需要保存其他任务的上下文

```
// 第二章：直接跳转到用户态
unsafe {
    __restore(KERNEL_STACK.push_context(...) as *const _ as usize);
}
```

第三章：引入 `__switch` 上下文切换

- 保存当前任务的内核栈指针和 callee-saved 寄存器
- 恢复下一个任务的上下文

```
// 第三章：上下文切换
unsafe extern "C" {
    pub unsafe fn __switch(
        current_task_cx_ptr: *mut TaskContext,
        next_task_cx_ptr: *const TaskContext,
    );
}
```

`switch.S` 关键代码：

```
__switch:
    # 保存当前任务的 sp 和 ra/s0-s11
    sd sp, 8(a0)
    sd ra, 0(a0)
    .set n, 0
    .rept 12
        SAVE_SN %n  # 保存 s0-s11
        .set n, n + 1
    .endr
    # 恢复下一个任务的 sp 和 ra/s0-s11
    ld ra, 0(a1)
    .set n, 0
    .rept 12
        LOAD_SN %n  # 恢复 s0-s11
        .set n, n + 1
    .endr
    ld sp, 8(a1)
    ret
```

三、系统调用变化

3.1 新增系统调用

系统调用	ID	功能	说明
`sys_yield`	124	主动让出 CPU	应用主动调用，触发协作式任务切换

3.2 sys_exit 实现变化

第二章：

```rust
pub fn sys_exit(exit_code: i32) -> ! {
    println!("[kernel] Application exited with code {}", exit_code);
    run_next_app()  // 直接运行下一个应用
}
```

第三章：

```rust
pub fn sys_exit(exit_code: i32) -> ! {
    println!("[kernel] Application exited with code {}", exit_code);
    exit_current_and_run_next();  // 标记当前任务退出并切换
}
```

3.3 新增 yield 机制

```
// 第三章新增
pub fn sys_yield() -> isize {
    suspend_current_and_run_next();  // 挂起当前任务并切换
    0
}
```

四、异常处理变化

第二章：异常导致 panic

```
Trap::Exception(Exception::StoreFault) => {
    println!("[kernel] PageFault in application, kernel killed it.");
    panic!("[kernel] Cannot continue!");
}
```

第三章：异常触发任务切换

```
Trap::Exception(Exception::StoreFault) => {
    println!("[kernel] PageFault in application, bad addr = {:#x}, ...",
stval, cx.sepc);
    panic!("[kernel] Cannot continue!");
}
// 注意：在 ch3-coop 中，异常仍然会 panic，但在之后的版本会改为任务切换
```

五、用户态应用变化

5.1 第二章测试应用

应用	功能
`00hello_world.rs`	打印 Hello World
`01store_fault.rs`	触发存储错误
`02power.rs`	计算幂
`03priv_inst.rs`	特权指令测试
`04priv_csr.rs`	CSR 寄存器测试
`05panic_test.rs`	Panic 测试

5.2 第三章测试应用

应用	功能
`00write_a.rs`	打印 A 字符并 yield
`01write_b.rs`	打印 B 字符并 yield
`02write_c.rs`	打印 C 字符并 yield

第三章的应用演示了**协作式多任务**：

- 每个应用循环打印字符并调用 `yield_()` 让出 CPU
- 多个应用交替执行，展示并发效果

```
// 第三章应用示例
fn main() -> i32 {
    for i in 0..HEIGHT {
        for _ in 0..WIDTH {
            print!("A");
        }
        println!(" [{}/{}]", i + 1, HEIGHT);
        yield_();  // 主动让出 CPU
    }
    0
}
```

六、配置常量提取

第二章： 常量定义在 `batch.rs` 内部

第三章： 提取到独立的 `config.rs`

```
// config.rs
pub const USER_STACK_SIZE: usize = 4096 * 2;
pub const KERNEL_STACK_SIZE: usize = 4096 * 2;
pub const MAX_APP_NUM: usize = 4;  // 从 16 改为 4
pub const APP_BASE_ADDRESS: usize = 0x80400000;
pub const APP_SIZE_LIMIT: usize = 0x20000;
```

七、总结

第二章特点

- 批处理系统：应用程序按顺序执行
- 单应用加载：一次只加载一个应用
- 简单异常处理：异常导致 panic
- 基础系统调用：write, exit

第三章特点

- **协作式多任务**：多个应用可以并发执行
- **任务管理**：引入 TCB、任务状态、任务管理器
- **上下文切换**：通过 `__switch` 实现任务切换
- **新增 sys_yield**：应用主动让出 CPU
- **批量预加载**：所有应用同时加载到不同内存区域

核心演进

从"一个接一个执行应用"进化到"多个应用协作式并发执行"，为第四章的抢占式调度打下了基础。